

2024 中国研究生创“芯”大赛·EDA 精英挑 战赛

作品设计报告

赛题十：针对模拟电路的电路图到网表自
动生成

参赛队 ID：EDA241030

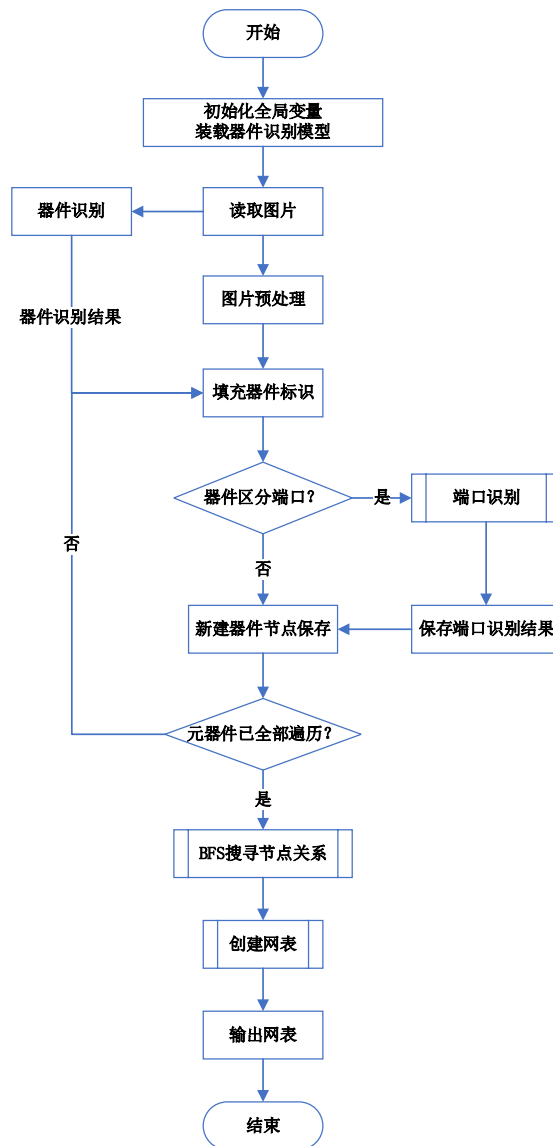
二〇二四年十一月

第一章 算法介绍

一、总体框架

赛题要求设计的算法需要以一组算例图片作为输入：png 格式的电路图，并以 Python 字典的形式作为输出：识别得到的电路功能结果 `ckt_type`，识别得到的电路元器件连接关系（网表）`ckt_netlist`。

算法整体思想是：对每个输入的图片，先对其所有元器件进行识别并归类；对所有已分类的电路元件，维护并关注其端口名称、数量和引出方向；对电路整体使用 BFS 算法寻找线路并建立网表，直至各元器件都被连接起来，得到最终结果，流程图如下所示：



二、变量定义与数据预处理

程序对于每个测例，定义并存储以下变量：

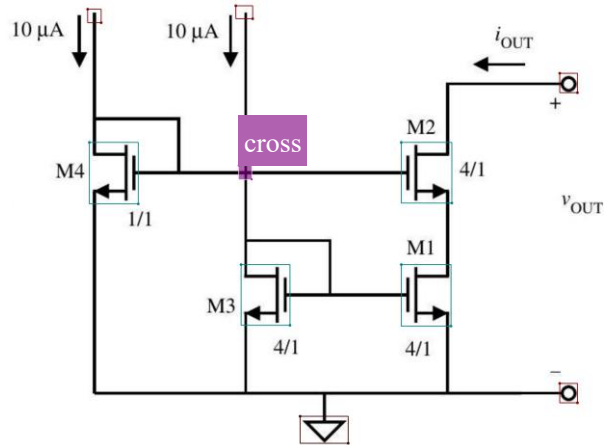
- 1) **info**: 列表。在读取图片时初始化为空，并在识别电路元件后存储整个电路图识别到的所有元件；
- 2) **nodes**: 列表。在读取图片时初始化为空，并在寻路过程中记录遍历的电路元件；
- 3) **nets**: 列表。在读取图片时初始化为空，并在寻路后存储整个电路图最终存在的网络。

程序对于单个测例中的每个电路元器件，定义并存储以下变量：

- 1) **next_node**: 字典，临时变量。在执行寻路算法时，程序将每个电路元件视为图（graph）中的一个节点（node），该变量与当前节点在同一网络（net）中的其他节点的编号，记录器件间的直接连接关系；
- 2) **next_net**: 字典。在寻路算法后创建网表时，会根据 **next_node** 中的连接关系，将互联的器件对应端口转换为 **next_net** 中的同一个网络编号，以此记录器件接口所属的逻辑网络；
- 3) **direction_to_port**: 字典。在调用端口识别后，记录特定器件端口引出的方向，包括上、下、左、右，用于建立网表时确定端口对应的网络编号。

程序开始时，上述变量初始化为空，并在算法过程中逐步构建。

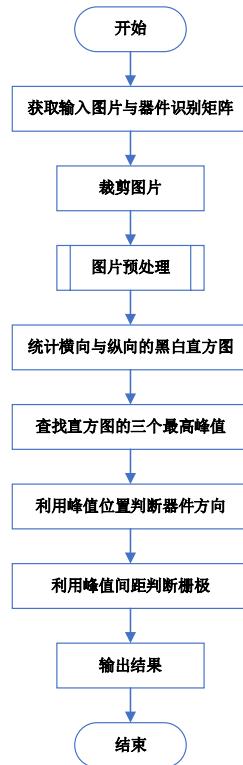
对于训练数据集，我们额外手动标注了 1,536 个“cross”标签，用于标识线路交叉点，如下图：



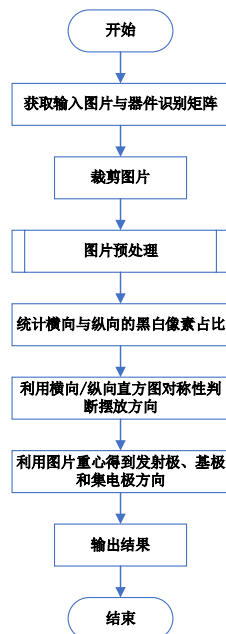
三、器件端口识别

对于需要考虑端口极性的器件，依据其类型分别调用对应的端口识别程序。例如，对于 MOS

器件（PMOS、NMOS）使用的端口识别步骤如下：



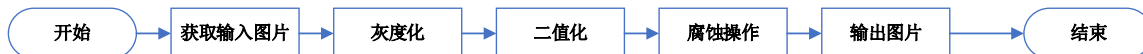
对于 BJT 器件（NPN、PNP）使用的端口识别步骤如下：



四、图片预处理

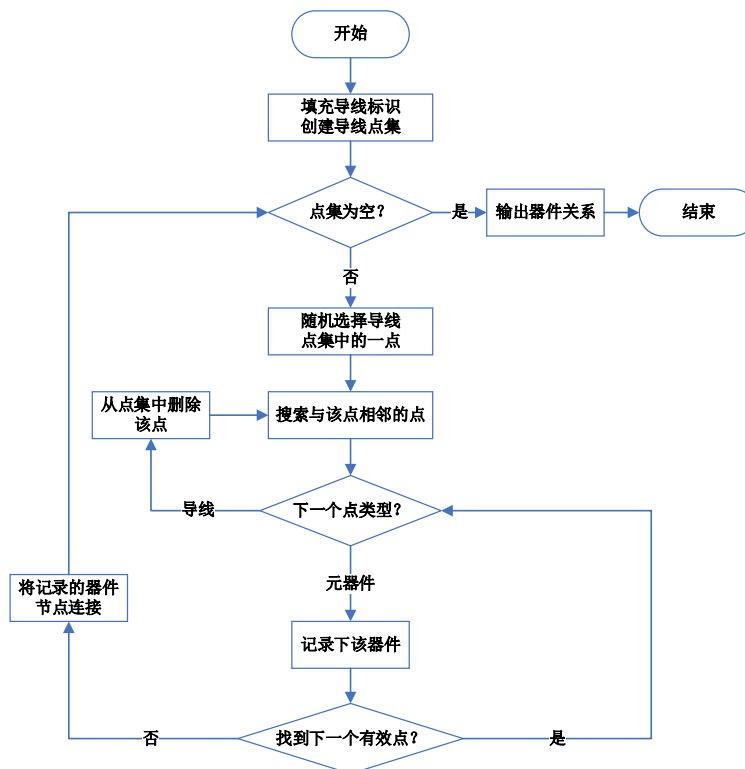
为了保证器件端口识别时，输入的裁剪后图片清晰度，减小图片颜色和分辨率对直方图数

据的影响，在识别器件端口以及后续的网络构建过程之前都会对图片进行以下预处理：



五、BFS 构建网络

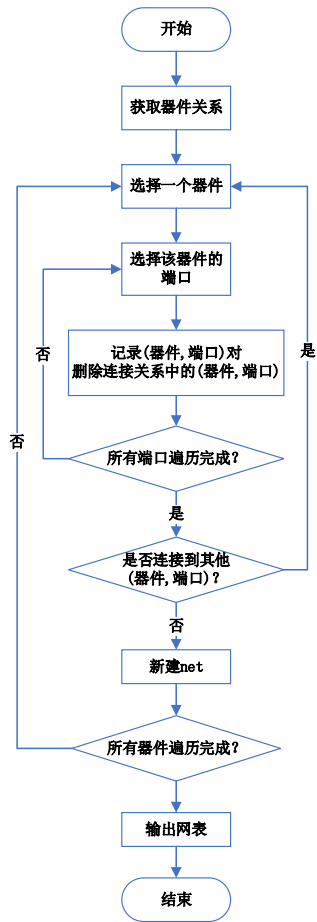
获取电路图中各元件连接关系构建网络，采用基于二值化矩阵的广度优先搜索（BFS）算法，流程图如下：



1. 首先，将所有导线点加入到导线点集，并新建一个队列用来存储 BFS 算法的当前状态；
2. 随机选择导线点集中的一个点，从该点开始，向上下左右四个方向依次遍历相邻点；
3. 如遇到的点是导线，则加入到队列中。如遇到的点是元器件，则记录下来；
4. 直到当前队列所有的点，即可认为记录的所有元器件都是连接，并把这种连接记录下来；
5. 如果点集中的点尚未完全被遍历，则重新清空队列，从步骤 2 重复运行算法。

六、网表生成

网络创建后，根据 node 和 net 对应信息，以及电路特征，生成最终网表，包含电路各器件连接关系，以及预测电路功能。流程图大致如下：



第二章 第三方库说明

下表列出了本次设计的算法程序安装并使用到的第三方库及其在程序中所涉功能的简要说明。

第三方库名称	版本号	功能
numpy	~=2.1.2	提供高效的多维数组对象及相关操作的数学函数库。
opencv-python	~=4.10.0.84	用于计算机视觉和图像处理的开源库，支持多种图像和视频处理功能。
pillow	~=10.2.0	提供简单易用的图像处理和图像文件格式转换功能的库。
matplotlib	~=3.9.2	用于创建静态、动态和交互式可视化图表的绘图库。
ultralytics	~=8.3.25	提供先进的 YOLO 模型及工具，用于快速实现目标检测任务。
scipy	~=1.14.1	为科学计算提供高效的数值算法和统计工具的库。
scikit-learn	~=1.5.2	提供简单高效的数据挖掘和数据分析工具的机器学习库。
networkx	~=3.2.1	用于创建、操作和研究复杂网络结构的图论分析库。
torch	~=2.4.1+cu124	提供深度学习模型构建和训练工具的开源机器学习库。

第三章 测试结果分析

实验拟对赛题方公布的 60 个计算数据集在本地进行计算生成网表，在服务器端进行网表验证，下面列出测试结果，包括各算例中电路功能识别准确度、网表识别准确度、程序运行时间：

测例编号	功能识别精准度 F	网表识别精准度 K	运行时间(s)
001	1	(46.0, 41.0)	6.619
002	1	(208.0, 208.0)	19.417
003	1	(15.0, 14.0)	6.504
004	1	(10.0, 10.0)	1.557
005	1	(88.0, 88.0)	6.902
006	1	(115.0, 115.0)	7.009
007	1	(48.0, 48.0)	3.289
008	1	(69.0, 69.0)	5.11
009	1	(14.0, 10.0)	4.536
010	1	(110.0, 110.0)	66.955
011	1	(65.0, 65.0)	14.446
012	1	(85.0, 82.0)	7.411
013	1	(160.0, 158.0)	27.821
014	1	(16.0, 16.0)	3.573
015	1	(5.0, 5.0)	1.431
016	1	(12.0, 12.0)	5.174
017	1	(54.0, 54.0)	7.108
018	1	(161.0, 160.0)	10.521
019	1	(9.0, 9.0)	1.882
020	1	(54.0, 54.0)	3.045
021	1	(47.0, 47.0)	8.384
022	0	(73.0, 73.0)	6.647
023	1	(141.0, 139.0)	6.522
024	1	(12.0, 12.0)	4.193
025	1	(9.0, 9.0)	3.82
026	1	(14.0, 14.0)	4.646
027	0	(54.0, 54.0)	4.256
028	1	(30.0, 30.0)	6.863
029	1	(63.0, 62.0)	4.578
030	1	(58.0, 56.0)	5.787
031	1	(42.0, 37.0)	5.081
032	1	(19.0, 18.0)	3.595
033	1	(61.0, 61.0)	6.959
034	1	(28.0, 20.0)	22.741

035	1	(73.0, 72.0)	3.842
036	1	(57.0, 56.0)	8.189
037	1	(12.0, 12.0)	6.076
038	1	(131.0, 131.0)	4.525
039	1	(143.0, 142.0)	7.757
040	1	(6.0, 6.0)	3.809
041	1	(70.0, 70.0)	7.846
042	1	(42.0, 42.0)	2.731
043	1	(103.0, 103.0)	4.026
044	1	(38.0, 34.0)	4.044
045	1	(71.0, 65.0)	5.785
046	0	(16.0, 16.0)	2.586
047	1	(6.0, 6.0)	34.427
048	1	(39.0, 39.0)	2.848
049	1	(42.0, 30.0)	5.984
050	1	(18.0, 14.0)	8.625
051	1	(176.0, 176.0)	11.107
052	1	(99.0, 99.0)	7.145
053	1	(76.0, 75.0)	5.299
054	1	(37.0, 35.0)	5.589
055	1	(44.0, 39.0)	6.017
056	1	(71.0, 70.0)	6.279
057	1	(11.0, 11.0)	5.67
058	1	(111.0, 111.0)	8.072
059	1	(101.0, 101.0)	9.459
060	1	(47.0, 42.0)	4.538

总功能识别精确度 F: 0.9500

总网表识别精确度 K (3s): 0.2808

总网表识别精确度 K (15min): 0.2815

总运行时间: 486.6570

上述测试数据仅为实验测试结果，最终结果以提交代码为准。

参考文献

- [1] Rautela, Deepak, and Rajendra Katti. "Design and implementation of FPGA router for efficient utilization of heterogeneous routing resources." IEEE Computer Society Annual Symposium on VLSI: New Frontiers in VLSI Design (ISVLSI'05). IEEE, 2005.
- [2] Salcedo-Sanz, Sancho, Yong Xu, and Xin Yao. "Meta-heuristic algorithms for FPGA segmented channel routing problems with non-standard cost functions." Genetic Programming and Evolvable Machines 6 (2005): 359-379.
- [3] McMurchie, Larry, and Carl Ebeling. "PathFinder: A negotiation-based performance-driven router for FPGAs." Proceedings of the 1995 ACM third international symposium on Field-programmable gate arrays. 1995.
- [4] Alexander, Michael J., et al. "Placement and routing for performance-oriented FPGA layout." VLSI Design 7.1 (1998): 97-110.
- [5] Haritaoğlu, Ismail, and Cevdet Aykanat. "A global routing heuristic for FPGAs based on mean field annealing." International Workshop on Field Programmable Logic and Applications. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994.